Green Balls: Emergent Anti-Collision Orbits

A Language-Agnostic Guide to Recreating the NoCol Simulation

This document reverse-engineers johnBuffer's NoCol — a C++/SFML simulation in which balls of varying sizes orbit a central point and, through nothing but three simple rules applied every frame, *discover* trajectories that never intersect. No pathfinding. No planning. No collision avoidance AI. Just emergence.

The goal of this guide is to distill the mechanism thoroughly enough that you can reimplement it in any language, rendering framework, or dimensional context (2D canvas, WebGL, Godot, Processing, a terminal with Unicode, whatever) and get the same mesmerizing result.

Table of Contents

1. The Effect and Why It's Interesting
2. Core Concepts
3. Data Model
4. Initialization
5. The Simulation Loop
6. Rule 1: Central Attraction
7. Rule 2: Overlap Correction (Not Collision)
8. Rule 3: Position Integration
9. Why This Converges — The Emergence Mechanism
10. Stability Detection and Visual Feedback
11. Trail Rendering
12. Tunable Parameters and Their Effects

13. Pseudocode Reference Implementation
14. Common Pitfalls
15. Extensions and Variations

---

1. The Effect and Why It's Interesting

When running, the simulation displays roughly 20 circles of varying radii (from tiny 5px dots to large 70px orbs) orbiting a shared center point on a black background. Initially, the balls overlap frequently and glow red. Over the course of seconds to minutes, they settle into stable, interlocking orbital paths where no two balls ever touch. They turn green. They leave fading trail lines behind them, producing intricate, layered, Spirograph-like patterns.

The visual is deeply satisfying for several reasons:

- **Near-miss tension.** Balls of wildly different sizes weave past each other with apparent millimeter precision. Your brain pattern-matches for collisions that never arrive.
- **Apparent intentionality.** The orbits look *designed* — choreographed, even — but they emerge from rules that have zero awareness of the global pattern.
- **Generative beauty.** The overlapping trail lines produce complex harmonic curves reminiscent of Lissajous figures or orbital resonance diagrams.

Most importantly: **there is no trajectory planning.** The system doesn't solve for collision-free paths. It stumbles into them through a process that is, at its core, a form of gradient descent on the space of possible orbits. Understanding *why* it converges is the most valuable part of this document.

---

2. Core Concepts

The entire simulation rests on three principles applied every frame:

#	Rule	What It Does	What It Affects
1	**Central attraction**	Pulls every ball gently toward the center	Velocity
2	**Overlap correction**	Pushes overlapping balls apart	Position only
3	**Position integration**	Moves balls along their velocity	Position

That's it. The magic is in what Rule 2 does *not* do: it does not modify velocity. This is the key to everything.

---

3. Data Model

Each ball needs the following properties:

```
Ball:
    position        : Vector2D       # Current (x, y)
    velocity        : Vector2D       # Current (vx, vy)
    radius          : Float          # Fixed at creation, varies per ball
    stable          : Boolean        # Was this ball collision-free this frame?
    stable_count    : Integer        # Consecutive frames of stability for this b
    position_history: CircularBuffer # Last N positions, for trail rendering
```

You also need:

```
Global:
    center          : Vector2D       # Center of the simulation space
    ok_count        : Integer        # Consecutive globally-stable frames
```

A note on the circular buffer

The position history is a fixed-size array (original uses 100 entries) with a write index that wraps around. Each frame, the current position overwrites the oldest entry. This gives you a sliding window of the last N positions for trail drawing without any allocation.

---

4. Initialization

Spawning positions

Balls are placed in a **ring** around the center, not scattered randomly across the canvas. This is important — it means they all start at roughly the same distance from the attractor, which helps them settle into orbital patterns rather than chaotic scatter.

```
For each ball:
    angle  = random(0, 2π)
    spawn_radius = 350.0    # Adjustable; roughly 1/3 of a 1080p half-height
    x = center.x + spawn_radius * cos(angle)
    y = center.y + spawn_radius * sin(angle)
```

Spawning velocities

Each component is an integer in [-4, +4), giving slow, randomized initial motion. The original code uses `rand() % 8 - 4`. Any small random velocity works — the exact distribution doesn't matter much because the system will reshape these trajectories.

```
vx = random_integer(-4, 3)  # or random_float(-4, 4)
vy = random_integer(-4, 3)
```

Spawning radii

Uniformly distributed between a minimum and maximum:

```
radius = random_float(min_size, max_size)
# Default: min_size = 5, max_size = 70
```

The large variance in radius is aesthetically important. It creates a visual hierarchy and makes near-misses between a huge ball and a tiny one particularly dramatic.

Initialize all history positions

Fill the entire position history buffer with the starting position so trails don't contain garbage data:

```
for i in 0..history_length:
    position_history[i] = (x, y)
```

---

5. The Simulation Loop

Each frame follows this exact order:

```
1. For each ball: mark stable = true, save position to history
2. Run the physics update (attraction + overlap correction)
3. Update global stability counter (ok_count)
4. Move all balls by their velocity (position integration)
5. Render
```

Steps 1–3 happen in the `update()` function. Step 4 is separate. This ordering matters. Let's break down each rule.

6. Rule 1: Central Attraction

```
For each ball:
    to_center = center - ball.position
    ball.velocity += ATTRACTION_STRENGTH * to_center
```

Where `ATTRACTION_STRENGTH = 0.01`.

What this does

This is a simple linear spring-like attraction: the force is proportional to displacement. It is NOT inverse-square gravity — it gets *stronger* the farther the ball is from the center, which prevents escape. Every ball is constantly being pulled inward.

Why the variable is called `attraction_force_bug`

The original code names this constant `attraction_force_bug` (0.01) alongside an unused `attraction_force` (50.0) and an unused `attractor_threshold` (50.0). This strongly suggests the original design had a more complex attractor (likely inverse-distance gravity with a threshold cutoff), and the simple linear version at 0.01 was either a bug or placeholder that happened to produce beautiful results. The author kept it.

Why this matters for convergence

The attractor serves two purposes:

1. **Confinement**: It prevents balls from drifting off-screen.
2. **Orbital generation**: The combination of velocity + a restoring force toward a point naturally produces elliptical/orbital motion. Without this, balls would just drift in straight lines.

Important: no damping

There is **no friction, drag, or velocity damping** anywhere in the system. Balls do not slow down over time. The only thing that modifies velocity is the central attraction. This is important — the system is approximately energy-conserving, which is part of why the orbits persist indefinitely once found.

---

7. Rule 2: Overlap Correction (Not Collision)

This is the heart of the simulation and the most important section of this document.

The algorithm

```
For each pair of balls (i, k) where i < k:
    delta = ball_i.position - ball_k.position
    distance = length(delta)
    min_distance = ball_i.radius + ball_k.radius

    if distance < min_distance:
        # They overlap. Push them apart.
        axis = delta / distance                          # Unit vector from k to
        overlap = min_distance - distance                # How much they overlap
        ball_i.position += 0.5 * overlap * axis          # Push i outward
        ball_k.position -= 0.5 * overlap * axis          # Push k outward

        # Mark both as unstable this frame
        ball_i.stable = false
        ball_k.stable = false
```

What this does NOT do

- It does **not** modify velocity.
- It does **not** compute elastic bounce angles.
- It does **not** exchange momentum.
- It does **not** apply impulses.

This is purely a **positional constraint resolution**. When two balls overlap, they are pushed apart by exactly the amount needed to make them tangent, split 50/50 between both balls. Their velocities are completely untouched.

Why this is not a “collision”

In a standard physics engine, when two circles overlap, you compute the collision normal, decompose velocities into normal and tangential components, apply coefficients of restitution, and redirect the balls. That produces bouncing.

This code does none of that. It’s closer to what you’d see in **Verlet integration** or **position-based dynamics** (PBD), where constraints are enforced by directly adjusting positions. In Verlet, this implicitly modifies velocity because velocity is derived from position differences. But in NoCol, velocity is stored and integrated *separately* — so the position correction creates a subtle one-frame displacement that doesn’t directly feed back into velocity.

The implicit effect on trajectory

Even though velocity isn't modified, the position correction *does* affect the trajectory. Here's why: after the overlap correction displaces a ball's position, the next frame's central-attraction calculation (`to_center = center - ball.position`) uses the **corrected** position. So the attraction force vector is slightly different than it would have been without the correction. Over many frames, these tiny angular nudges accumulate into a meaningful reshaping of the orbit.

Think of it this way: the overlap correction doesn't redirect the ball like a bounce. It *displaces* the ball, which changes the geometry of its attraction to the center, which bends its orbit slightly. It's an indirect, second-order effect.

---

8. Rule 3: Position Integration

```
dt = 0.016   # Fixed timestep (~60fps equivalent)

For each ball:
    ball.position += dt * ball.velocity
```

This is straightforward Euler integration. The position moves along the velocity vector. In slow-motion mode, the original divides dt by a `speedDownFactor` to effectively sub-step the movement.

Order of operations note

Position integration happens **after** the physics update. This means:

1. Balls are first pushed apart if overlapping (modifying position).
2. Then balls move along their velocity (modifying position again).

The attraction modifies velocity during the physics update, and that modified velocity is used here. So within a single frame: attraction affects velocity → overlap affects position → integration affects position.

---

9. Why This Converges — The Emergence Mechanism

This is the central question: **why does a system with no planning find collision-free orbits?**

The short answer

The overlap correction acts as a **soft optimization step** that nudges orbits away from configurations that produce overlaps, without disrupting the orbital structure. Because the system is approximately energy-conserving and the correction is gentle, it doesn't scatter orbits chaotically — it prunes them incrementally. Over many corrections, only orbits that are *already compatible* survive unmodified, and orbits that conflict get gradually reshaped until they're compatible too.

The longer answer

Consider what happens to two balls whose orbits cross:

1. At some point, they'll be near the crossing point at the same time and overlap.
2. The overlap correction pushes them apart — displacing both of their positions by a small amount perpendicular to their current velocities.
3. This displacement changes their position relative to the center.
4. On the next frame, the attraction force vector is slightly different (because their position is different), which bends their velocity slightly.
5. This bend changes the shape of their orbit by a small amount.
6. The next time they approach the crossing point, they arrive at a slightly different time or slightly different position.

If the new orbits still cross, the correction happens again, nudging them further. If they don't cross, the correction doesn't happen, and the orbits persist. This is effectively a **stochastic hill-climbing process** on the space of orbit configurations, where the "cost function" is overlap frequency and the "gradient step" is the position correction.

Why it doesn't diverge

Three properties prevent the system from exploding:

1. **The correction is symmetric and minimal.** Each ball moves by exactly half the overlap distance. No energy is added. The correction is the smallest displacement that resolves the overlap.
2. **The attractor provides a basin.** Without central attraction, corrected balls would drift away. The attractor recaptures them into orbital motion, so corrections result in slightly different orbits rather than scatter.
3. **Velocity is not touched.** In a bouncing simulation, collisions redirect velocity, which can create new collisions (cascade). Here, velocity is preserved, so the orbital "shape"

changes gradually rather than abruptly.

Why it's not guaranteed

The system doesn't *always* converge, and it can take varying amounts of time. With too many balls, too large radii, or too small a simulation space, there may not be enough room for non-overlapping orbits to exist. The original defaults (20 balls, radii 5–70, 1920×1080 space) are tuned to a sweet spot where convergence is reliable but not instant.

An analogy

Imagine people walking in circles around a maypole at different distances and speeds, in a crowded space. When two people get too close, they each take a small step away from each other. They don't stop, change direction, or negotiate — they just shift over slightly. Over the course of minutes, everyone unconsciously adjusts their paths to avoid each other, settling into a stable pattern. Nobody *planned* the pattern. It emerged from the cumulative effect of minimal local avoidance, contained by the shared attractor (the maypole).

---

10. Stability Detection and Visual Feedback

Per-ball stability

Each frame, before the physics update, every ball's `stable` flag is set to `true`. If the overlap check finds that ball involved in a collision, it's set to `false`. After the update:

```
For each ball:
    if ball.stable:
        ball.stable_count += 1
    else:
        ball.stable_count = 0
```

Global stability

A global `ok_count` variable tracks consecutive frames where **no** overlaps occurred across the entire system:

```
if any_overlap_occurred:
    if ok_count < 200:
        ok_count = 0    # Note: only resets if not yet "locked in"
else:
    ok_count += 1
```

The threshold of 200 acts as a confidence window. Once `ok_count` exceeds 199, the system is considered reliably stable, and a single spurious overlap (due to floating-point imprecision) won't reset it.

Color feedback

Ball color is a linear interpolation between red (unstable) and green (stable):

```
if ok_count > 199:
    stable_ratio = 1.0                              # Globally stable: force all
else:
    stable_ratio = min(1.0, ball.stable_count / 255.0)  # Per-ball fade

color = stable_ratio * GREEN + (1.0 - stable_ratio) * RED
```

So a ball that hasn't collided in 255+ frames will be fully green even if the *global* system hasn't reached the 200-frame threshold yet. Once `ok_count` crosses 200, all balls are forced green simultaneously.

This two-tier system gives you a nice visual narrative: individual balls find their orbits and turn green one by one, and then eventually the whole system "clicks" into place.

---

11. Trail Rendering

Each ball maintains a circular buffer of its last 100 positions. Each frame, the current position is saved:

```
ball.position_history[ball.write_index] = ball.position
ball.write_index = (ball.write_index + 1) % HISTORY_LENGTH
```

Drawing the trail

The trail is drawn as a connected line strip (polyline), reading from the buffer starting at `write_index` (the oldest point) and wrapping around to `write_index - 1` (the newest point).

```
For i in 0..HISTORY_LENGTH:
    actual_index = (i + write_index) % HISTORY_LENGTH
    opacity = i / HISTORY_LENGTH      # 0.0 at oldest, 1.0 at newest
    point = position_history[actual_index]
    color = opacity * GREEN           # Fades from black to green
    draw_line_segment_to(point, color)
```

Blend mode

The original uses **additive blending** for trails. This means overlapping trail lines become brighter rather than occluding each other, which is essential for the layered luminous look. Where many orbits cross, you get bright hotspots. On a black background, additive blending is particularly striking.

If your rendering framework supports blend modes, use additive blending for the trail layer. If not, you can approximate it by drawing semi-transparent lines on a dark background and accepting that you'll lose the glow-overlap effect.

---

12. Tunable Parameters and Their Effects

Parameter	Default	Effect of Increasing	Effect of Decreasing
`ATTRACTION_STRENGTH`	0.01	Tighter, faster orbits; quicker convergence; less graceful motion	Wider, lazier orbits; balls may drift far; slower convergence
`objects_count`	20	Harder to converge; more visual complexity; may never stabilize	Trivially easy to converge; less interesting
`min_size`	5	Fewer tiny balls to weave between gaps	Very small balls pass between larger ones easily (helps convergence)
`max_size`	70	Large balls dominate space; harder to find paths	Everything is small; less dramatic near-misses
`spawn_radius`	350	Balls start farther out; take longer to settle	Balls start clumped; more initial chaos
`dt` (integration)	0.016	Faster motion; potential instability	Slower motion; more sub-steps needed
`HISTORY_LENGTH`	100	Longer trails; more visual complexity	Shorter trails; snappier look

Sweet spot guidelines

For a satisfying result, aim for:

- Ball count where the space feels "busy but not packed" — roughly 15–30 for a 1920×1080 canvas.
- Radius range with at least a 5:1 ratio between max and min for visual variety.
- Attraction strength that produces orbital periods of a few seconds (too fast looks frantic; too slow looks static).

---

13. Pseudocode Reference Implementation

This is a complete, framework-agnostic pseudocode of the simulation. Implement this in your chosen language/framework.

```
CONSTANTS:
    WIDTH = 1920
    HEIGHT = 1080
    CENTER = (WIDTH / 2, HEIGHT / 2)
    NUM_BALLS = 20
    MIN_RADIUS = 5.0
    MAX_RADIUS = 70.0
    SPAWN_RADIUS = 350.0
    ATTRACTION = 0.01
    DT = 0.016
    HISTORY_LEN = 100
    STABILITY_THRESHOLD = 200

# ── Data structures ──

struct Ball:
    pos: Vec2
    vel: Vec2
    radius: Float
    stable: Bool
    stable_count: Int
    history: Vec2[HISTORY_LEN]
    hist_idx: Int

# ── Initialization ──

balls = []
for i in 0..NUM_BALLS:
```

```
    angle = random_float(0, 2 * PI)
    x = CENTER.x + SPAWN_RADIUS * cos(angle)
    y = CENTER.y + SPAWN_RADIUS * sin(angle)
    r = random_float(MIN_RADIUS, MAX_RADIUS)
    vx = random_float(-4, 4)
    vy = random_float(-4, 4)

    ball = Ball(
        pos = (x, y),
        vel = (vx, vy),
        radius = r,
        stable = true,
        stable_count = 0,
        history = [(x, y)] * HISTORY_LEN,
        hist_idx = 0
    )
    balls.append(ball)

ok_count = 0

# ── Main Loop (every frame) ──

function frame():

    # 1. Reset stability flags and save positions
    for ball in balls:
        ball.stable = true
        ball.history[ball.hist_idx] = ball.pos
        ball.hist_idx = (ball.hist_idx + 1) % HISTORY_LEN

    # 2. Apply central attraction (modifies velocity)
    for ball in balls:
        to_center = CENTER - ball.pos
        ball.vel += ATTRACTION * to_center

    # 3. Overlap correction (modifies position only)
    any_overlap = false
    for i in 0..len(balls):
        for k in (i+1)..len(balls):
            delta = balls[i].pos - balls[k].pos
            dist = length(delta)
            min_dist = balls[i].radius + balls[k].radius

            if dist < min_dist and dist > 0:
                any_overlap = true
                balls[i].stable = false
                balls[k].stable = false
```

```
            axis = delta / dist
            correction = 0.5 * (min_dist - dist)
            balls[i].pos += correction * axis
            balls[k].pos -= correction * axis

# 4. Update stability counters
for ball in balls:
    if ball.stable:
        ball.stable_count += 1
    else:
        ball.stable_count = 0

if any_overlap:
    if ok_count < STABILITY_THRESHOLD:
        ok_count = 0
else:
    ok_count += 1

# 5. Integrate position (modifies position)
for ball in balls:
    ball.pos += DT * ball.vel

# 6. Render
clear_screen(BLACK)

# Draw trails (additive blend)
for ball in balls:
    for i in 0..HISTORY_LEN - 1:
        idx_a = (i + ball.hist_idx) % HISTORY_LEN
        idx_b = (i + 1 + ball.hist_idx) % HISTORY_LEN
        opacity = i / HISTORY_LEN
        draw_line(
            ball.history[idx_a],
            ball.history[idx_b],
            color = GREEN * opacity,
            blend = ADDITIVE
        )

# Draw balls
for ball in balls:
    if ok_count > STABILITY_THRESHOLD - 1:
        ratio = 1.0
    else:
        ratio = min(1.0, ball.stable_count / 255.0)
    color = lerp(RED, GREEN, ratio)
    draw_circle(ball.pos, ball.radius, color)
```

14. Common Pitfalls

Pitfall: Modifying velocity during overlap correction

If you accidentally add impulses or bounce velocities when balls overlap, the system will bounce chaotically instead of settling. The entire convergence mechanism depends on velocity being left untouched during overlap correction. **This is the single most important thing to get right.**

Pitfall: Using inverse-square gravity

If you use `1/distance²` attraction instead of linear (`distance * constant`), balls far from the center feel almost no pull and can escape. Linear attraction acts as a spring that *always* brings them back. If you want to use inverse-square, add a strong damping or containment boundary.

Pitfall: Distance-of-zero division

If two balls are spawned at the exact same position, `delta / dist` is a division by zero. Guard against this:

```
if dist < min_dist and dist > 0:
    ...
```

If `dist == 0` , either skip the pair or push them apart along an arbitrary axis.

Pitfall: Variable timestep

If your dt fluctuates frame-to-frame, the attraction force accumulates differently, which can make orbits unstable. Use a fixed timestep for the physics, even if your rendering framerate varies. The original sidesteps this by using a constant `dt = 0.016` .

Pitfall: Too many balls or too large radii

If the sum of ball diameters approaches the available orbital space, convergence becomes impossible — there's literally no room for non-overlapping orbits. Keep the total "ball area" well below the "ring area" around the center.

Pitfall: Forgetting additive blending on trails

Without additive blending, trails will just look like a mess of opaque green lines. The visual

magic depends on trail overlaps reinforcing each other into brighter regions.

---

15. Extensions and Variations

Once you have the basic system working, here are directions to explore:

Multiple attractors. Replace the single center with two or more attractor points. Balls will form figure-eight or more complex orbits as they're pulled between them.

3D. The algorithm generalizes trivially to three dimensions — just use Vec3 for position and velocity. The overlap correction math is identical (push apart along the vector between centers). Rendering trails as 3D ribbons could be spectacular.

Variable attraction strength. Make attraction proportional to ball mass (radius³ or radius²). Larger balls orbit tighter; smaller ones orbit wider. This could create a more "solar system" feel.

Obstacles. Add fixed circles that balls must avoid (using the same overlap correction, but the obstacle doesn't move). This constrains the orbits further and could produce more intricate patterns.

Attraction to neighbors instead of center. Replace the central attractor with mutual gravitational attraction between all balls. This turns it into an n-body system with soft collisions, which may produce clustering or binary-orbit pairing.

Musical mapping. The original repo includes sound files. You could map overlap events to sounds (percussive hits when balls collide during the settling phase, silence during stability) or map orbital frequency to pitch.

Evolution / genetic approach. Instead of letting the system settle naturally, you could record orbit parameters at stability and use them as seeds for the next generation — selecting for faster convergence or more aesthetically complex trail patterns.

Asymmetric correction. Instead of 50/50 push, push smaller balls more and larger balls less (proportional to inverse mass). This would let large balls maintain more stable orbits while small balls adapt around them.

Summary of the Core Insight

The NoCol simulation demonstrates that **collision avoidance can emerge from collision correction** — that if you gently fix overlaps without disrupting momentum, a confined system of orbiting bodies will iteratively discover non-overlapping trajectories. No

intelligence, no planning, no optimization objective. Just three local rules, applied millions of times, producing global order.

The mechanism is:

1. Attraction creates orbits.
2. Overlap correction nudges orbits incrementally.
3. Because velocity is preserved, orbits reshape gradually rather than scatter.
4. Orbits that don't conflict persist. Orbits that do conflict get nudged until they don't.
5. Stability is an attractor in the dynamical-systems sense: once reached, the system stays there.

This is the same deep principle underlying flocking algorithms, crystal lattice formation, and orbital resonance in real solar systems: **local rules, iterated sufficiently, produce global structure.**